EECS 482
Introduction to operating systems

Segmentation and paging

Peter Chen
University of Michigan

Segmentation: translation

Segment #	Base	Bound	Description
3	2000	1000	stack segment
2	n/a	0	unused
1	0	500	data segment
0	4000	700	code segment

- Physical address for virtual address `(3, 100)` ?
  - `2000 + 100  = 2100`
- Physical address for virtual address `(0, ff)` ?
  - `4000 + ff = 40ff`
- Physical address for virtual address `(1, 2000)` ?
  - invalid (illegal) ➔ segmentation violation/core dump
- Physical address for virtual address `(2, ff)` ?
  - invalid (illegal) ➔ segmentation violation/core dump

Valid vs. invalid addresses

- Not all virtual addresses are valid
  - Valid → address is part of virtual address space
  - Invalid → virtual address is illegal to access
    - Accessing invalid address causes trap to OS

- In segmentation, a virtual address can be invalid due to:
  - Out of bound offset
  - Invalid segment number

Segmentation: protection

Segment #	Base	Bound	Description	Protection
3	2000	1000	stack segment	read/write
2	n/a	0	unused	
1	0	500	data segment	read/write
0	4000	700	code segment	read

Segmentation: translation algorithm

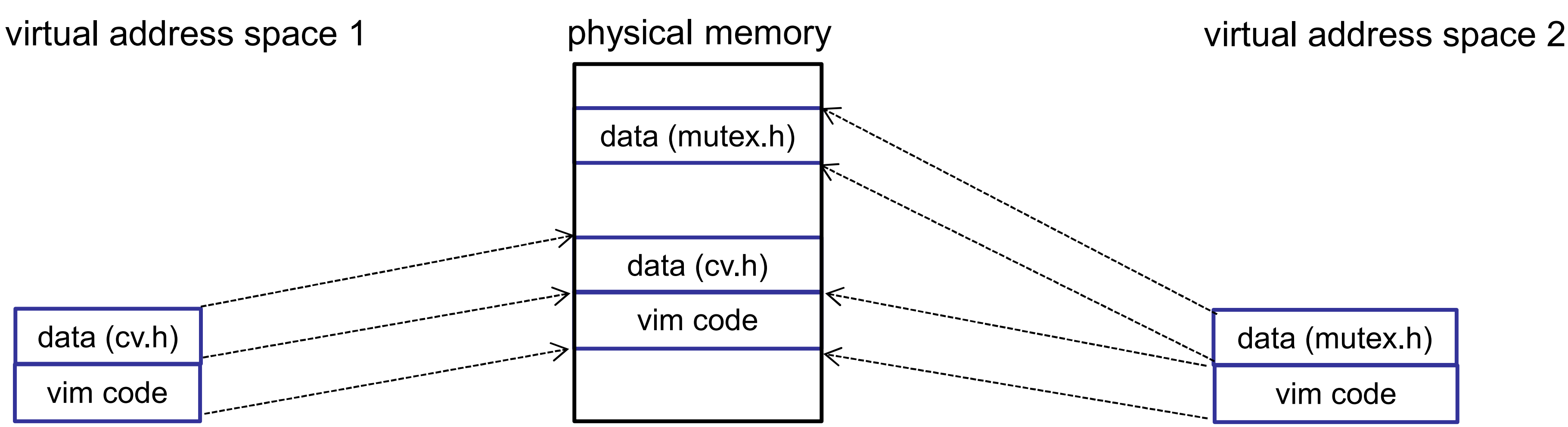
```
(base, bound, protection) = segment_info[segment]

if (segment is invalid or protected, or offset >= bound) {
        trap to kernel
} else {
        physical address = offset + base
}
```

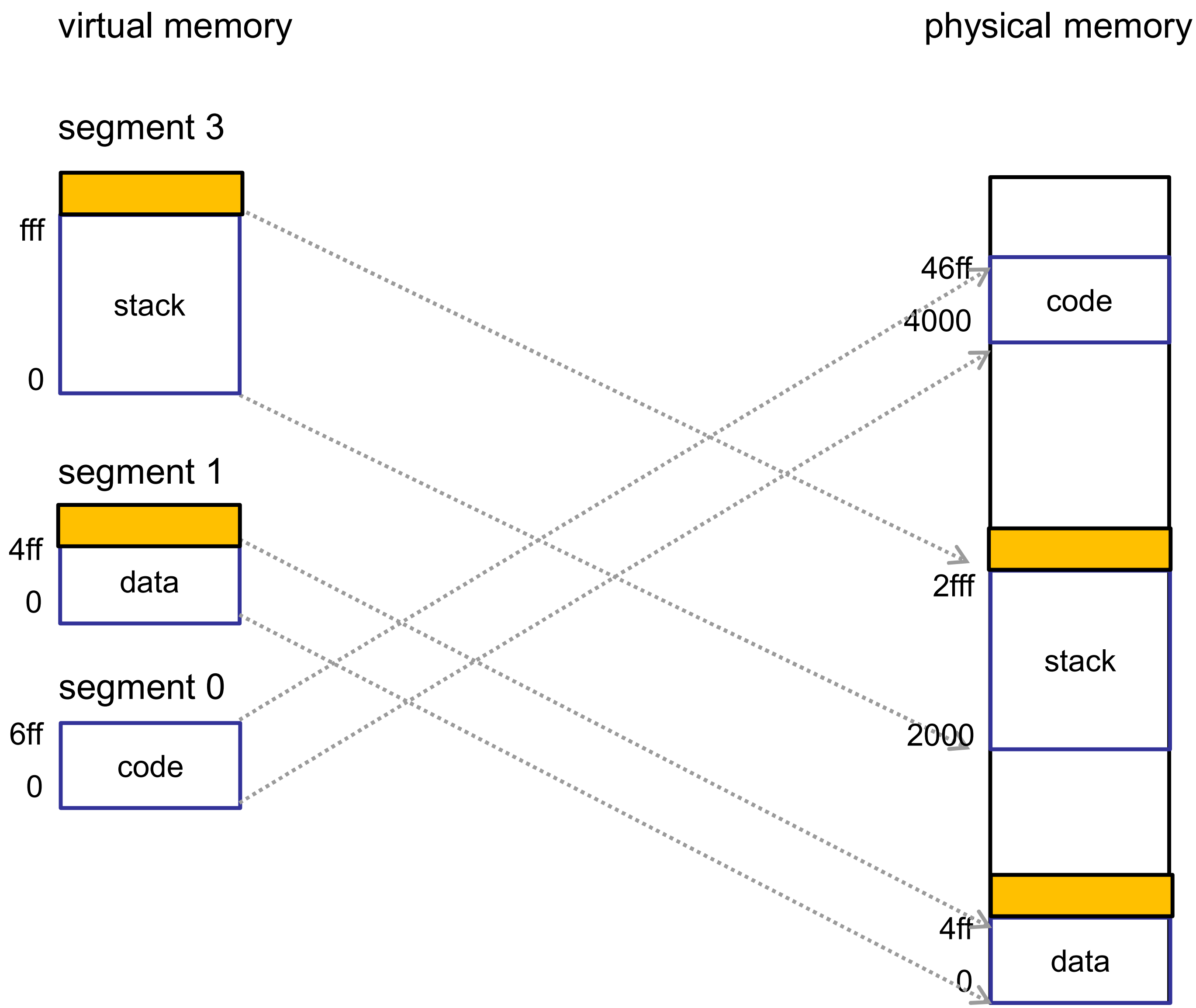
- What must be changed to switch address spaces?

Segment #	Base	Bound	Description	Protection
3	2000	1000	stack segment	read/write
2	n/a	0	unused	
1	0	500	data segment	read/write
0	4000	700	code segment	read

Segmentation supports partial sharing



Segmentation allows multiple growing regions

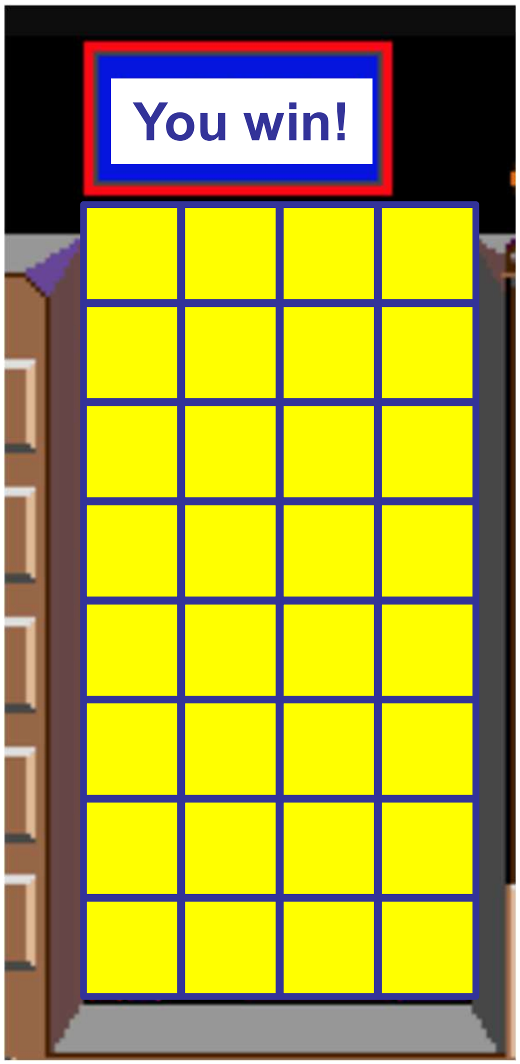


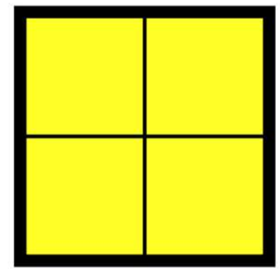
Segmentation: problems

- Growing address space still slow (may need to copy segment)
- External fragmentation
- Does not support large address spaces?
  - Each segment must fit in memory

- How can we:
  - Make memory allocation easy
  - Not have to worry about external fragmentation
  - Allow address space to be larger than physical memory

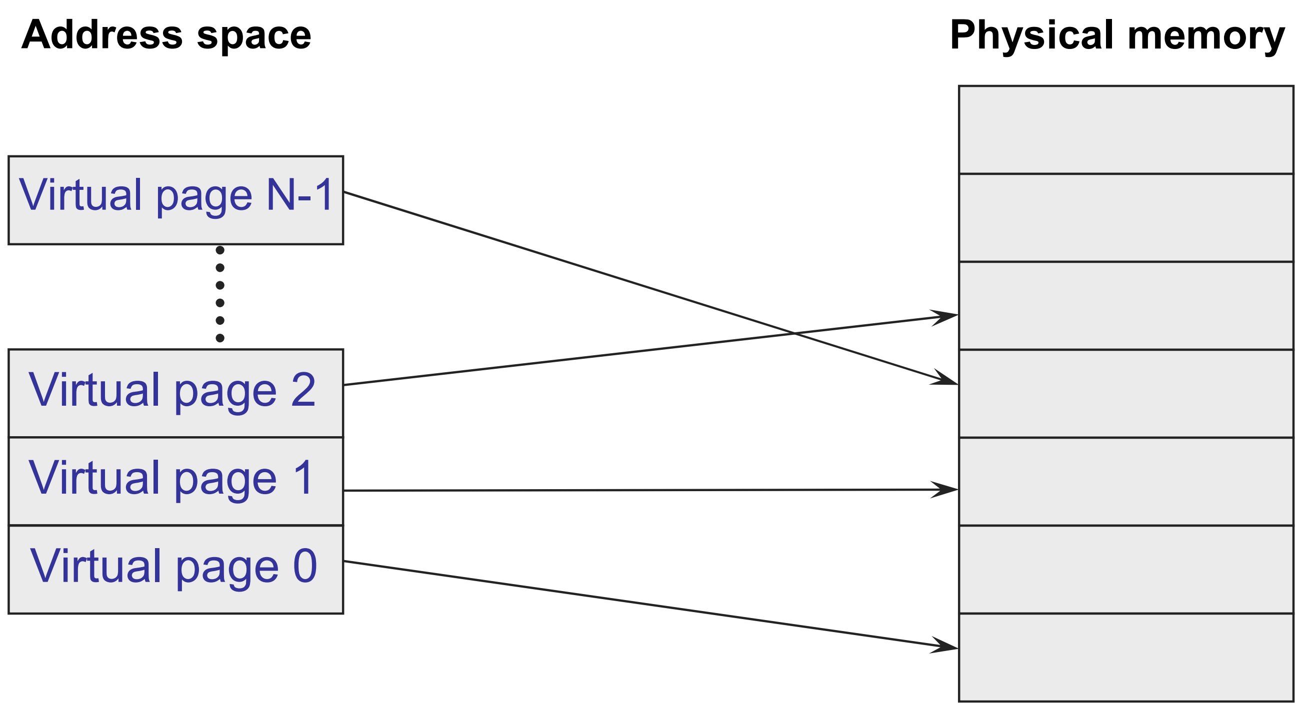
Tetris analogy





Paging

- Allocate memory in fixed-size units (pages)
  - Any free physical page can store any virtual page



Segmentation → paging

Virtual page # ~~Segment #~~	Physical page # Base / Page Size	Bound	Valid
3	4~~000~~	~~700~~ 1000	1
2	n/a	~~0~~ 0	0
1	1b~~000~~	~~2000~~ 1000	1
0	2~~000~~	1000	1

Page
~~Segment~~ table

Segmentation → paging

- Virtual address is of the form: (~~segment~~ page #, offset)

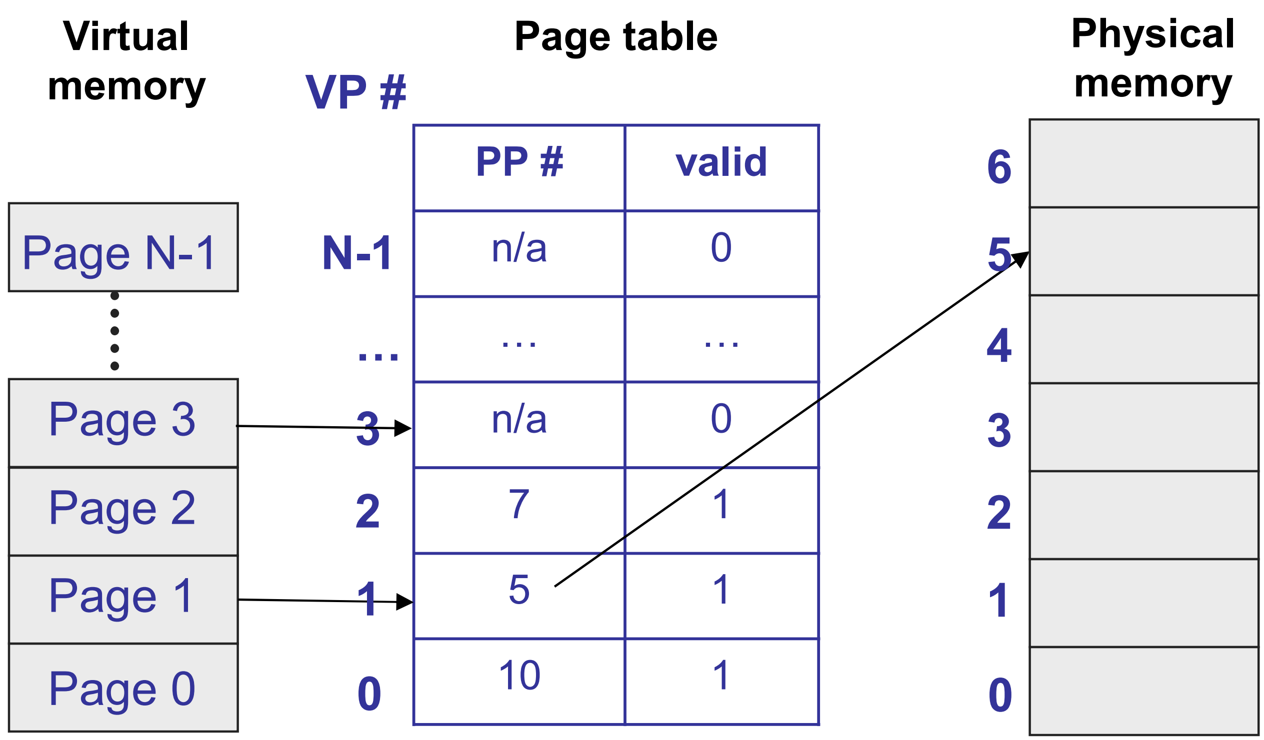
Example (32-bit address, 4096 byte page)

31 12	11 0
Page number	**Offset**

Page table

Virtual page #	Physical page #	Valid
1048575	n/a	0
…	n/a	0
3	n/a	0
2	7	1
1	5	1
0	10	1

Paging: translation




```
if (virtual page is invalid) {
       trap to OS fault handler
} else {
       physical page # = pageTable[virtual page #].physPageNum
       physical address = {physical page #}{offset}
}
```

Page table

- Lots of entries in page table, so store in memory
  - e.g., 32-bit address space, 4 KB pages ➔ 1M entries in page table
- To change address spaces, must change entire page table!
- How to speed this up?

Page table base register (PTBR)

Page table

PP #	valid
n/a	0
…	…
n/a	0
n/a	0
7	1
5	1
10	1

Page table: additional information

PP #	valid	protection	resident
n/a	0		0
…	…	…	…
n/a	0		0
20	1		1
7	1	rw	1
5	1		0
10	1	r	1

- Each page can be readable and/or writable
- Each page can be resident (in memory) or non-resident (on disk)

Paging: MMU algorithm

```
if (virtual page is invalid or protected or non-resident)
        trap to OS fault handler; OS kills process or retries
                                        access
} else {
        physical page # = pageTable[virtual page #].physPageNum
        physical address = {physical page #}{offset}
}
```

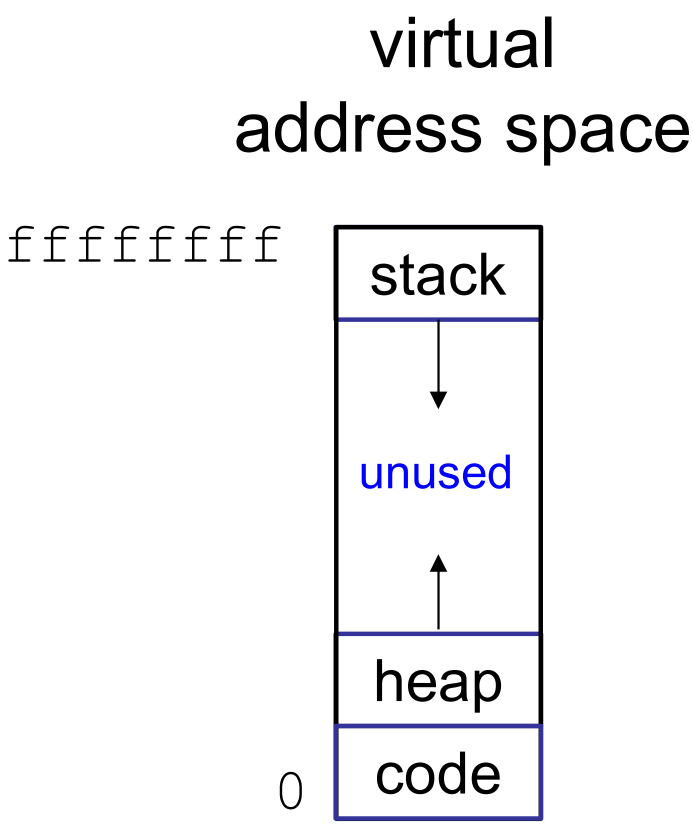
Valid versus resident

PP #	valid	protection	resident
n/a	0		0
…	…	…	…
n/a	0		0
20	1		1
7	1	rw	1
5	1		0
10	1	r	1

- Who makes a virtual page resident/non-resident?
- Who makes a virtual page valid/invalid?
- Why would a process want one of its virtual pages to be invalid?

- Valid → virtual page is legal for process to access. Access to invalid page is an error.
- Resident → virtual page is in physical memory. Access to non-resident page is not an error.

Sparse address spaces



Does this waste space (as it did with base and bound)?

What size should a page be?

- What happens if page size is really small?
- What happens if page size is really big?

- Typically a compromise, e.g., 4 KB or 8 KB
  - Some architectures support multiple page sizes

Paging

- Pros
  - Easy memory allocation
  - Easy to grow address space
  - Flexible sharing
  - No physical memory used for reserved space in sparse address spaces
- Cons
  - Page table size
  - e.g., 32-bit virtual address, 4 KB pages, 4 byte PTEs
  - 2^{32} bytes of address space ➔ 1M pages ➔ 4 MB page table!
- How to reduce space needed for page table?

Multi-level paging

- Standard page table is a simple array
- Multi-level paging generalizes this into a tree

level 1 page table

0	1	2	3
	invalid	invalid	

level 2 page tables

	physical page #	valid, …
3	10	1, …
2	15	0, …
1	20	1, …
0	2	1, …

	physical page #	valid, …
3	30	0, …
2	4	1, …
1	8	1, …
0	3	1, …

Multi-level paging

Single-level paging

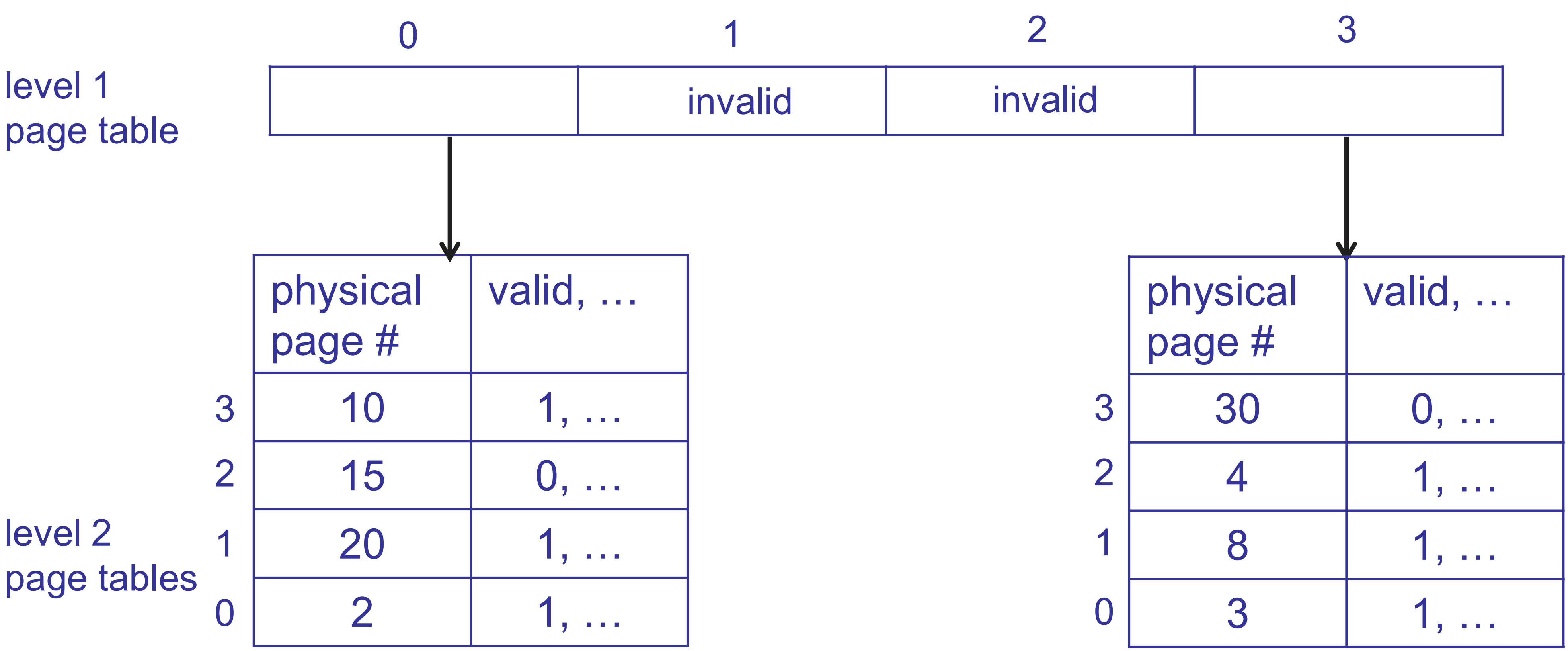
31 12	11 0
Page number	Offset

Multi-level paging

31 22	21 12	11 0
Index into level 1 PT	Index into level 2 PT	Offset

Multi-level paging

- How does this reduce size of translation data?
- Does it always reduce size of translation data?



Multi-level paging

- How to share memory between address spaces?
- What must be changed on a context switch?

Page table base register (PTBR)

level 1 page table

0	1	2	3
	invalid	invalid	

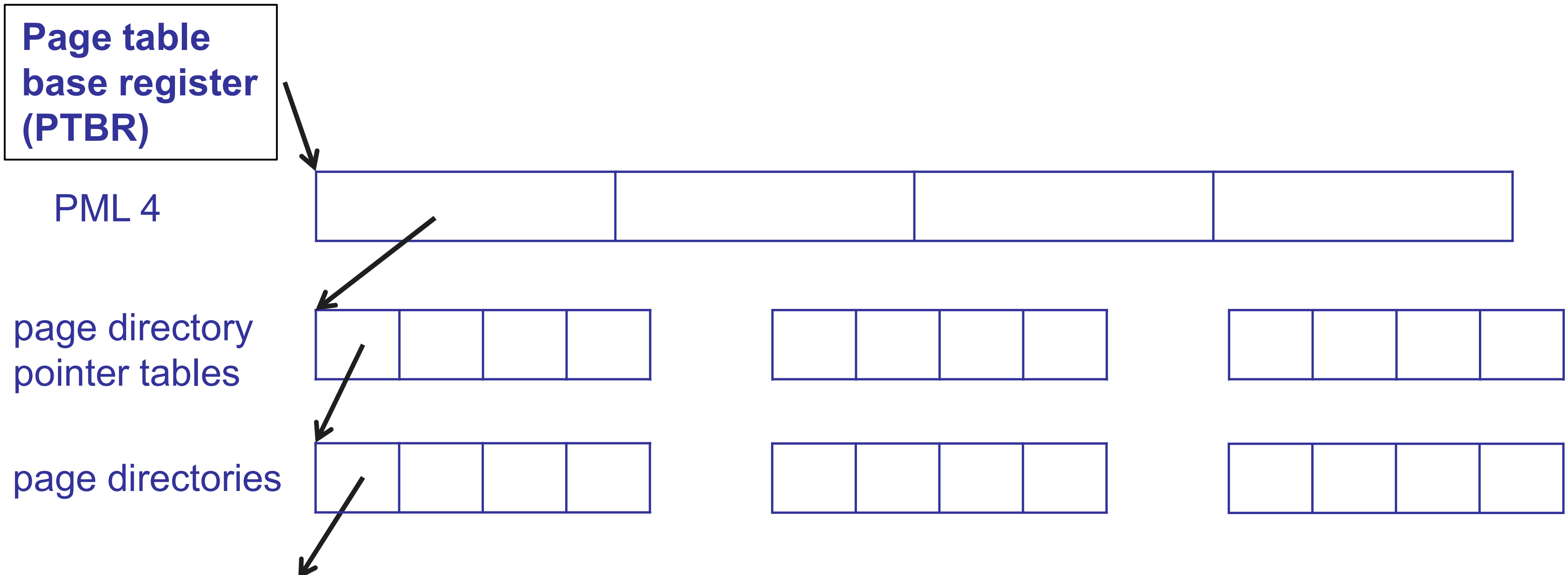
level 2 page tables

	physical page #	valid, …
3	10	1, …
2	15	0, …
1	20	1, …
0	2	1, …

	physical page #	valid, …
3	30	0, …
2	4	1, …
1	8	1, …
0	3	1, …

x86_64

47–39	38–30	29–21	20–12	11–0
index into PML 4	index into page directory pointer table	index into page directory	index into page table	offset



page tables

	physical page #	valid, …
3	10	1, …
2	15	0, …
1	20	1, …
0	2	1, …

	physical page #	valid, …
3	30	0, …
2	4	1, …
1	8	1, …
0	3	1, …

Multi-level paging

- Pros
  - Easy memory allocation
  - Easy to grow address space
  - Flexible sharing
  - Less translation data used for sparse address spaces

- Cons?

- Solution?

Translation lookaside buffer (TLB)

- TLB: a cache for page table entries
  - TLB hit → Skip all the translation steps
  - TLB miss → Get PTE, store in TLB, restart instruction
    - x86, ARM: done by hardware
    - MIPS: done by software

- Must invalidate TLB on context switch, or identify entries with address space ID

Pro tip: indirection + caching

- Indirection: solves all problems in CS!
  - Except performance ☹

- Caching: solves the performance problems caused by indirection
  - Pointers added by indirection are small, so are cached easily

Caching and virtual memory

- CPU caches are a cache for …?
- The TLB is a cache for …?
- Physical memory is a cache for ...?

- How do the concepts from CPU caches (EECS 370) relate to the concepts of virtual memory?
  - CPU caches: direct mapped, set associativity, tags
  - VM: virtual and physical addresses, page tables
  - Are these related?

CPU caching and virtual memory

- What placement algorithm (associativity) did you use in EECS 370?

- What associativity is used in VM?
  - Why?

CPU caching and virtual memory

- In EECS 370, how did you find a page in the set?
- Will this work for virtual memory?
- How would you solve this in EECS 281?